



Disciplina: Sistemas Distribuídos

Professora: Ana Cristina Barreiras Kochem Vendramin

Avaliação (valor 2,0)

Microserviços, Sistema de Mensageria, Criptografia de Chave Assimétrica

Desenvolver o *backend* de um **sistema distribuído de e-commerce** baseado em **microserviços** para gerenciamento de **pedidos, estoque de produtos, pagamentos, entregas e notificações**. O sistema deve seguir uma **arquitetura orientada a eventos (Event-Driven Architecture)**, na qual os processos se comunicam **exclusivamente através de eventos publicados e consumidos em um broker RabbitMQ**.

Cada microserviço deverá atuar de forma **independente e desacoplada**. Então, **não é permitido realizar chamadas diretas entre quaisquer processos**.

RabbitMQ

Criem duas *exchanges*: eCommerce (tipo *Direct*) e Promoções (tipo *Topic*). Não é permitido o uso de exchange *fanout*. Os eventos devem utilizar **routing keys hierárquicas**, permitindo que consumidores se inscrevam em diferentes categorias de eventos utilizando padrões de *binding*. Cada consumidor deve possuir sua própria fila.

(0,2) Processos Consumidores de Promoções

Processos consumidores de promoções apenas receberão notificações sobre promoções de produtos. Executam dois processos consumidores: o consumidor C1 registrará interesse nas categorias de produtos A e B e o consumidor C2 registrará interesse em todas as categorias. Cada consumidor deve criar sua própria fila e associá-la às *routing keys* correspondentes às categorias de interesse. Atenção: esses processos não podem realizar chamadas para nenhum dos microserviços. Eles devem se comunicar exclusivamente com o RabbitMQ, consumindo eventos de promoções.

(1,8) Backend do sistema de E-commerce

O *backend* deve ser implementado utilizando **5 microserviços independentes**, cada um responsável por uma parte da lógica da aplicação. As responsabilidades desses microserviços e os eventos publicados e/ou consumidos por cada um estão a seguir.

(0,5) Microserviço Principal

(0,2) Responsável por realizar a interação com os usuários por meio do terminal. Ele apresenta as opções do sistema e permite que os usuários executem ações como:

- visualizar produtos;
- realizar pedidos;

- excluir pedidos;
- consultar seus pedidos e respectivos status.

Cada novo pedido recebido deverá ser publicado como um evento no RabbitMQ, utilizando a *routing key* `pedido.criado`. Esse evento deverá conter, no mínimo, o identificador do pedido, os produtos, quantidades e informações necessárias para o processamento do pedido.

(0,3) O microserviço Principal deverá consumir os eventos:

- `pagamento.aprovado`;
- `pagamento.recusado`;
- `pedido.enviado`;
- `pedido.estoque_ok`;
- `estoque.indisponivel`.

A partir desses eventos, o microserviço principal deverá atualizar o status dos respectivos pedidos.

Quando um produto não estiver disponível em estoque ou quando o pagamento de um pedido for recusado, o microserviço Principal deverá publicar um evento utilizando a *routing key* `pedido.excluido`.

(0,3) Microserviço Estoque

O **Microserviço Estoque** é responsável pelo gerenciamento do estoque dos produtos.

O serviço deverá consumir os eventos:

- `pedido.criado`;
- `pedido.excluido`.

Ao receber um evento `pedido.criado`, o microserviço Estoque deverá verificar a disponibilidade dos produtos solicitados.

Caso todos os produtos estejam disponíveis, o serviço deverá realizar a respectiva reserva/baixa no estoque e publicar um evento utilizando a *routing key* `pedido.estoque_ok`, indicando que o pedido está apto a prosseguir para o processamento do pagamento.

Caso algum produto não esteja disponível, o microserviço Estoque deverá publicar um evento utilizando a *routing key* `estoque.indisponivel`, informando o pedido que não pode ser atendido.

Ao receber um evento `pedido.excluido`, o microserviço Estoque deverá devolver ao estoque os produtos que haviam sido reservados para o pedido.

(0,2) Microsserviço Pagamento

O **Microsserviço Pagamento** é responsável pelo processamento dos pagamentos dos pedidos.

O serviço deverá consumir o evento `pedido.estoque_ok`. Ao receber esse evento, deverá iniciar o processo de pagamento para o pedido correspondente.

O processamento do pagamento deverá ser simulado por meio do uso de **variáveis aleatórias** para determinar se o pagamento será aprovado ou recusado.

Quando o pagamento for aprovado, o microsserviço Pagamento deverá publicar um evento utilizando a *routing key* `pagamento.aprovado`.

Quando o pagamento for recusado, deverá publicar um evento utilizando a *routing key* `pagamento.recusado`.

(0,1) Microsserviço Entrega

O **Microsserviço Entrega** é responsável pelo gerenciamento da emissão de notas e da entrega dos produtos.

O serviço deverá consumir o evento `pagamento.aprovado`.

Após receber esse evento, deverá realizar as operações necessárias para emissão da nota e preparação da entrega. Após o processamento, deverá publicar um novo evento utilizando a *routing key* `pedido.enviado`, informando que o pedido foi enviado.

(0,2) Microsserviço Promoções

O **Microsserviço Promoções** é responsável pela geração e publicação de promoções de produtos. O serviço deverá gerar promoções aleatórias de produtos e publicá-las no RabbitMQ, utilizando *routing keys* que indiquem a categoria do produto, como `promocao.categoria.A`, `promocao.categoria.B`, `promocao.categoria.C`.

(0,5) Criptografia de Chave Assimétrica

(0,25) Assinatura digital dos eventos

Cada microsserviço que **publicar um evento** deverá:

1. gerar um *hash* do conteúdo do evento;
2. assinar o evento utilizando **criptografia assimétrica**;
3. incluir a assinatura digital no campo `Signature` do envelope do evento.

A assinatura deverá ser gerada utilizando a **chave privada do microsserviço produtor**.

(0,25) Validação da assinatura

Todo microserviço que **consumir um evento** deverá:

1. obter a chave pública do microserviço produtor;
2. verificar a assinatura digital do evento;
3. confirmar a autenticidade e a integridade da mensagem;
4. processar o evento **somente se a assinatura for válida**.

Caso a assinatura seja inválida, o evento deverá ser descartado.

Os microserviços devem possuir as chaves públicas de todos os demais microserviços. Criem uma pasta para cada microserviço e salvem as chaves públicas nessas pastas.

A figura 1 ilustra os eventos e os processos *publishers* e *subscribers*. A figura 2 ilustra a comunicação indireta entre os processos.

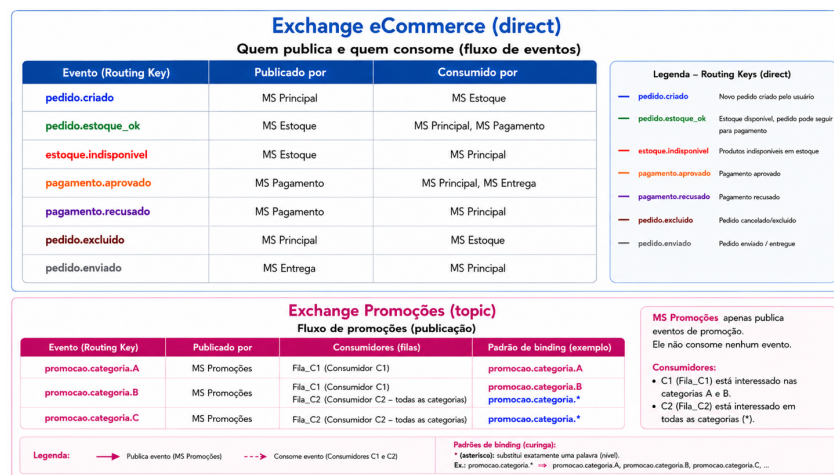


Figura 1 – Exchanges, routing keys, publishers e subscribers

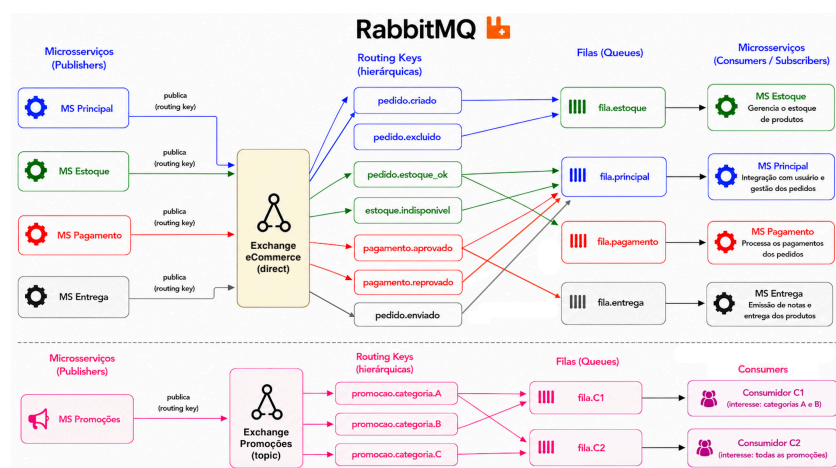


Figura 2 – Comunicação indireta entre os processos

Observações:

- O desenvolvimento do sistema deve ser em dupla.
- É obrigatória a defesa da aplicação para obter a nota.